Claude Code (Beginner → Advanced)

 **charliehills.substack.com**/p/claude-code-beginner-advanced

Charlie Hills, Sabahudin Murtic 🎁, Walid Boulanouar March 22, 2026

This is a collaborative issue. I cover the beginner setup. Sabahudin Murtic walks you through intermediate workflows with parallel agents for real client delivery. Walid Boulanouar shows what happens when Claude Code runs your entire agency. Three levels, one newsletter.

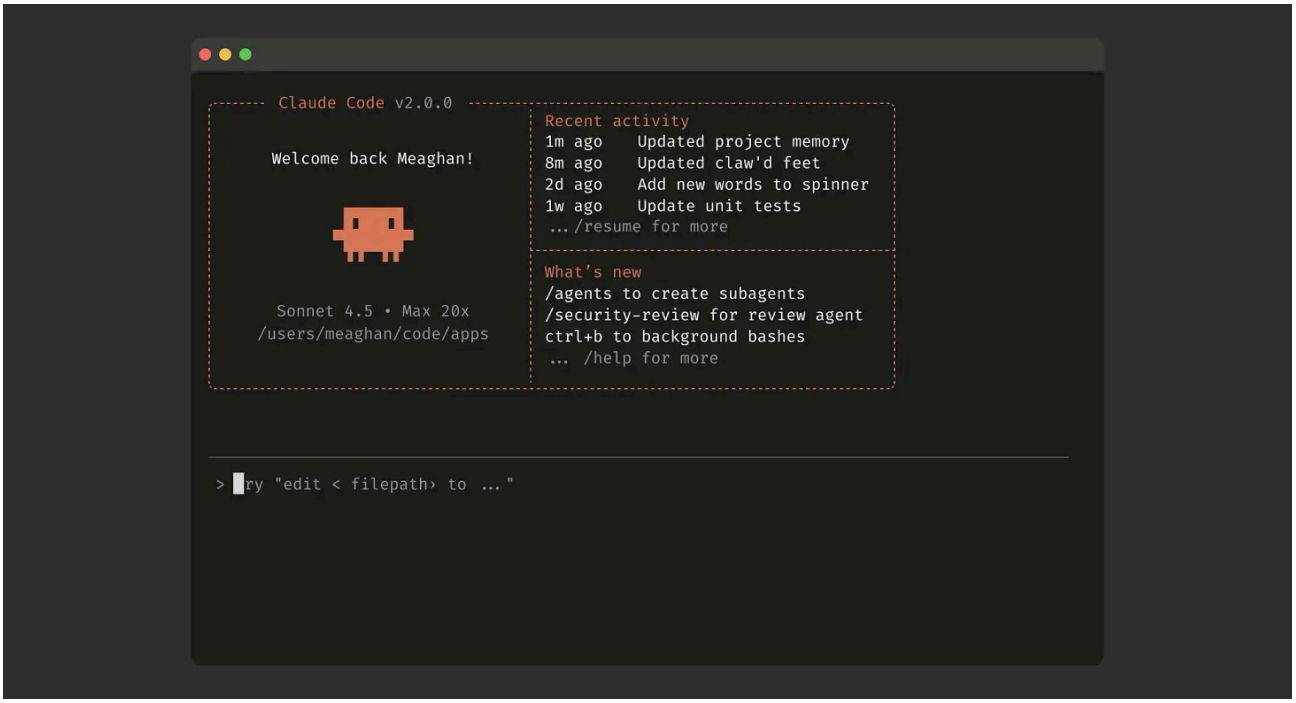
Everything in this newsletter is tested. I ran every workflow, hit every error, and documented what worked. If you are new here, welcome. If you have been reading for a while, thank you. Your subscription keeps this going.

Claude Code

Cowork is built on Claude Code. So is Claude's coding environment. So are most of the serious AI automation workflows producing real business results.

I am not a Claude Code expert. I learnt most of what follows while researching and writing this newsletter. I came at it as a marketer, not a developer. The barrier around tools like this is largely gone. A few hours of learning-by-doing get you real results.

What Claude Code actually is



A command-line tool you install in your terminal. It runs locally on your machine. It reads, writes, creates, and modifies files directly. It calls APIs. It browses the web. It spawns sub-

agents to run research in parallel. It executes Python scripts you have never written.

Think of it as an AI that lives inside your computer and works with your tools, not around them.

You do not have to be a developer to use it.

How to Claude Code (Beginner)

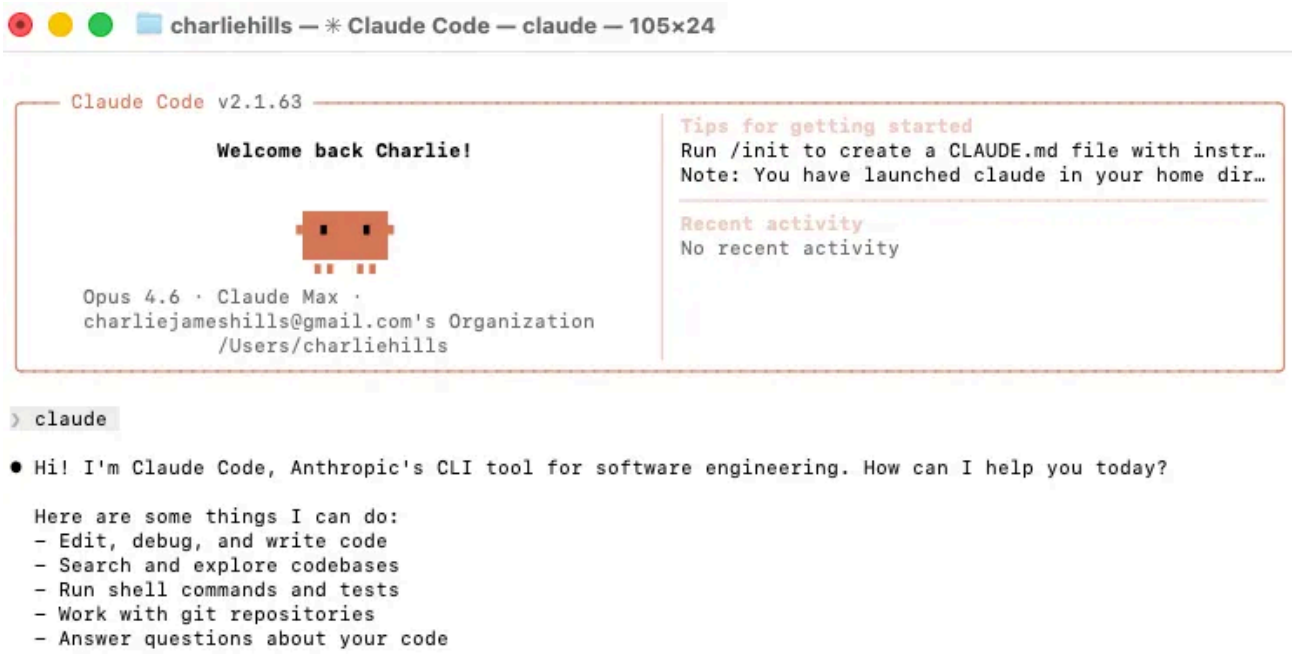
Getting set up

Step 1. Get a paid Claude plan. You need at least Pro at $20/month. Claude Max gives more usage for heavy workflows.

Step 2. Install Claude Code. Open your terminal. On Mac, open Spotlight and type “terminal.” On Windows, use CMD or PowerShell. Run the installation command from code.claude.com/docs. Takes about two minutes.

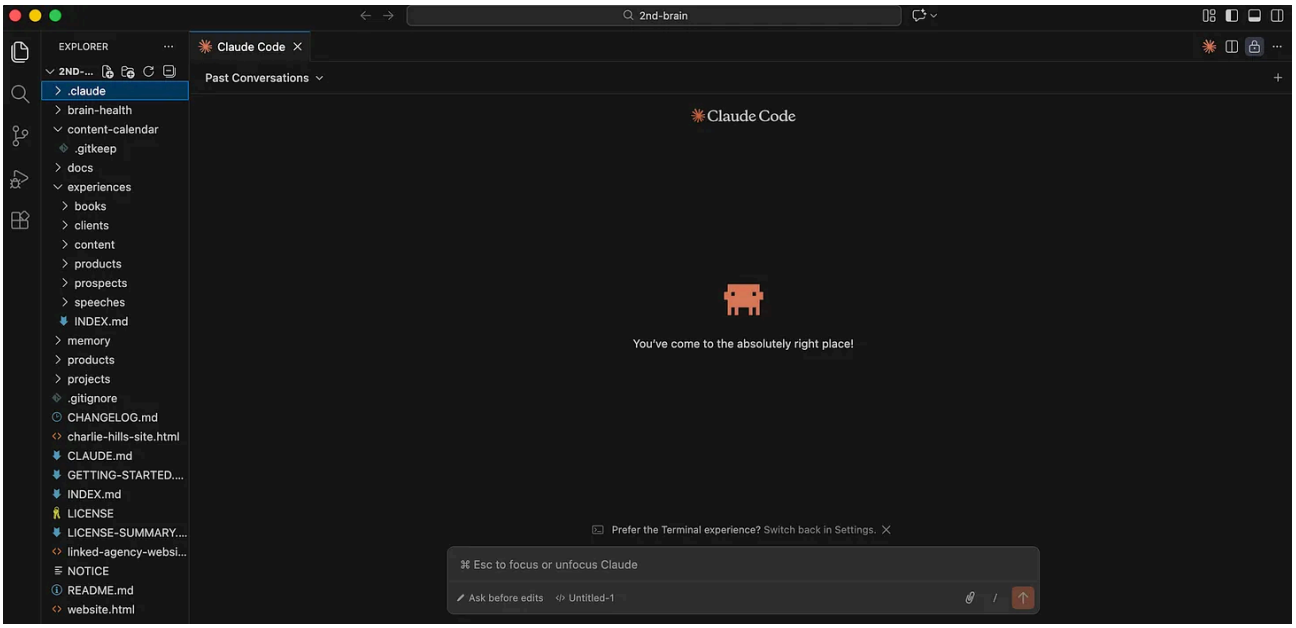
Step 3. Authenticate. Type “claude” in your terminal. It will prompt you to log in with your Claude account. Follow the steps.

Step 4. Navigate to a project folder. In your terminal, go to whatever folder you want Claude Code to work in. Type “claude” again. You are running.



If the terminal is new to you, use Claude Code through an IDE instead. The two recommended options are VS Code (free, from Microsoft, at code.visualstudio.com) and Windsurf (newer, more AI-native). Both let you run Claude Code through a graphical chat panel on the right while your files sit on the left. No raw terminal required.

To add Claude Code to VS Code: go to the Extensions tab, search “Claude Code,” install the one from Anthropic, and click the Claude icon that appears in your editor panel.



Free option (with tradeoffs):

You can run Claude Code with local open-source models through Ollama at zero cost.

Install Ollama, pull a model like qwen3-coder or gpt-oss:20b, and point Claude Code at your local endpoint.

No API fees or data leaving your machine.

You need at least 32GB of RAM for a usable experience. The output quality is noticeably lower than Claude's own models.

Good for learning the interface. Not enough for the multi-step workflows later in this newsletter.

Set up guide at docs.ollama.com/integrations/claude-code.

The CLAUDE.md file

This is the single highest-leverage thing you can do in Claude Code.

The CLAUDE.md file is a text file that sits in your project folder. Claude Code reads it automatically at the start of every session. It is injected into the context before you type a single message.

Workflow Orchestration

1. Plan Mode Default

- Enter plan mode for ANY non-trivial task (3+ steps or architectural decisions)
- If something goes sideways, STOP and re-plan immediately – don't keep pushing
- Use plan mode for verification steps, not just building
- Write detailed specs upfront to reduce ambiguity

2. Subagent Strategy

- Use subagents liberally to keep main context window clean
- Offload research, exploration, and parallel analysis to subagents
- For complex problems, throw more compute at it via subagents
- One task per subagent for focused execution

3. Self-Improvement Loop

- After ANY correction from the user: update `tasks/lessons.md` with the pattern
- Write rules for yourself that prevent the same mistake
- Ruthlessly iterate on these lessons until mistake rate drops
- Review lessons at session start for relevant project

4. Verification Before Done

- Never mark a task complete without proving it works
- Diff behavior between main and your changes when relevant
- Ask yourself: "Would a staff engineer approve this?"
- Run tests, check logs, demonstrate correctness

5. Demand Elegance (Balanced)

- For non-trivial changes: pause and ask "is there a more elegant way?"
- If a fix feels hacky: "Knowing everything I know now, implement the elegant solution"
- Skip this for simple, obvious fixes – don't over-engineer
- Challenge your own work before presenting it

6. Autonomous Bug Fixing

- When given a bug report: just fix it. Don't ask for hand-holding
- Point at logs, errors, failing tests – then resolve them
- Zero context switching required from the user
- Go fix failing CI tests without being told how

Task Management

1. **Plan First**: Write plan to `tasks/todo.md` with checkable items
2. **Verify Plan**: Check in before starting implementation
3. **Track Progress**: Mark items complete as you go
4. **Explain Changes**: High-level summary at each step
5. **Document Results**: Add review section to `tasks/todo.md`
6. **Capture Lessons**: Update `tasks/lessons.md` after corrections

Core Principles

- **Simplicity First**: Make every change as simple as possible. Impact minimal code.
- **No Laziness**: Find root causes. No temporary fixes. Senior developer standards.
- **Minimal Impact**: Changes should only touch what's necessary. Avoid introducing bugs.

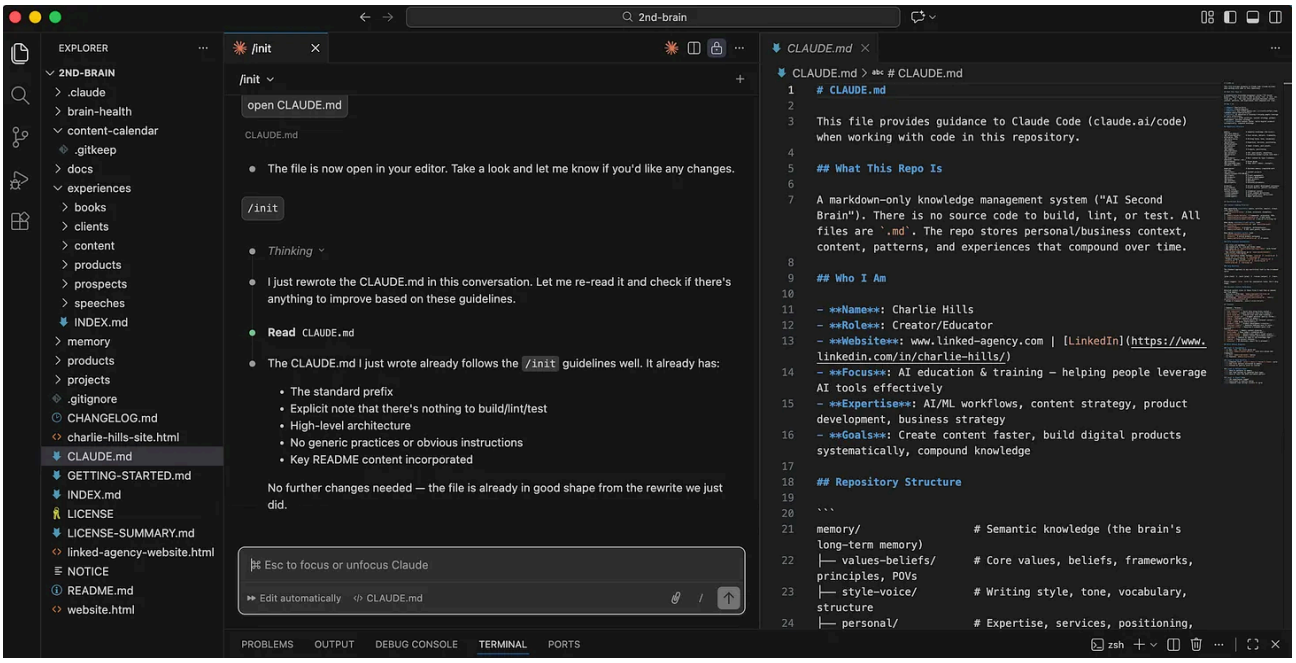
Think of it as the standing brief for this project. It tells Claude Code what the project is for, how you want it to behave, what it should never do, the tools and conventions you use, and any constraints specific to this work.

Every session with Claude Code starts cold. No memory of last time. The CLAUDE.md is what makes it feel like a tool that knows your work rather than a blank slate you brief every morning.

Your first CLAUDE.md does not need to be elaborate. Run `/init` in any project folder and Claude Code will read through your files and write one automatically. It analyses what is in the folder and produces a summary with working instructions. Then you refine it over time.

Rules for a good CLAUDE.md:

1. Keep the most important constraints at the very top. Claude Code has primacy bias. The first things it reads stick hardest.
2. Write in bullet points and short headings. High information density beats prose.
3. 200 to 500 lines maximum. Longer than that and you are paying token costs that reduce output quality.
4. Never paste entire API documentation into it. Give Claude Code a summary and let it fetch the rest when required.
5. Update it when Claude Code keeps making the same mistake. If it trips on the same issue twice, add a rule.



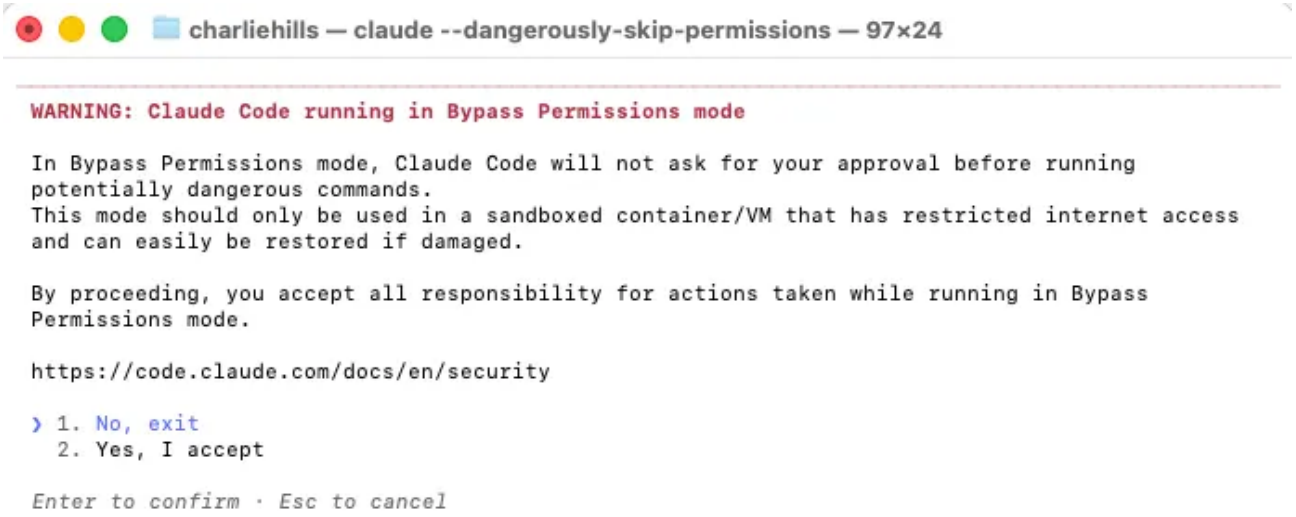
Permission modes

Claude Code has four modes. Understanding them changes how you use it.

Ask Before Edits is the default. Claude Code asks permission before changing any file. Safe, but slow. If it is asking you to approve every individual edit during a complex build, switch modes.

Edit Automatically auto-accepts changes to existing files but still asks before creating new ones. A good middle ground for most work.

Bypass Permissions removes all prompts. Claude Code works without interruption. To enable it, add the flag `--dangerously-skip-permissions` when launching Claude Code from your terminal. Type `claude --dangerously-skip-permissions` instead of just `claude`.



This is where most experienced users end up for knowledge work. The risk is real but low if you work in a non-critical folder. There was a widely shared incident where someone running bypass permissions on a Linux machine lost data. The command was valid. The context was wrong. Use with awareness.

Plan Mode is the one most beginners undervalue.

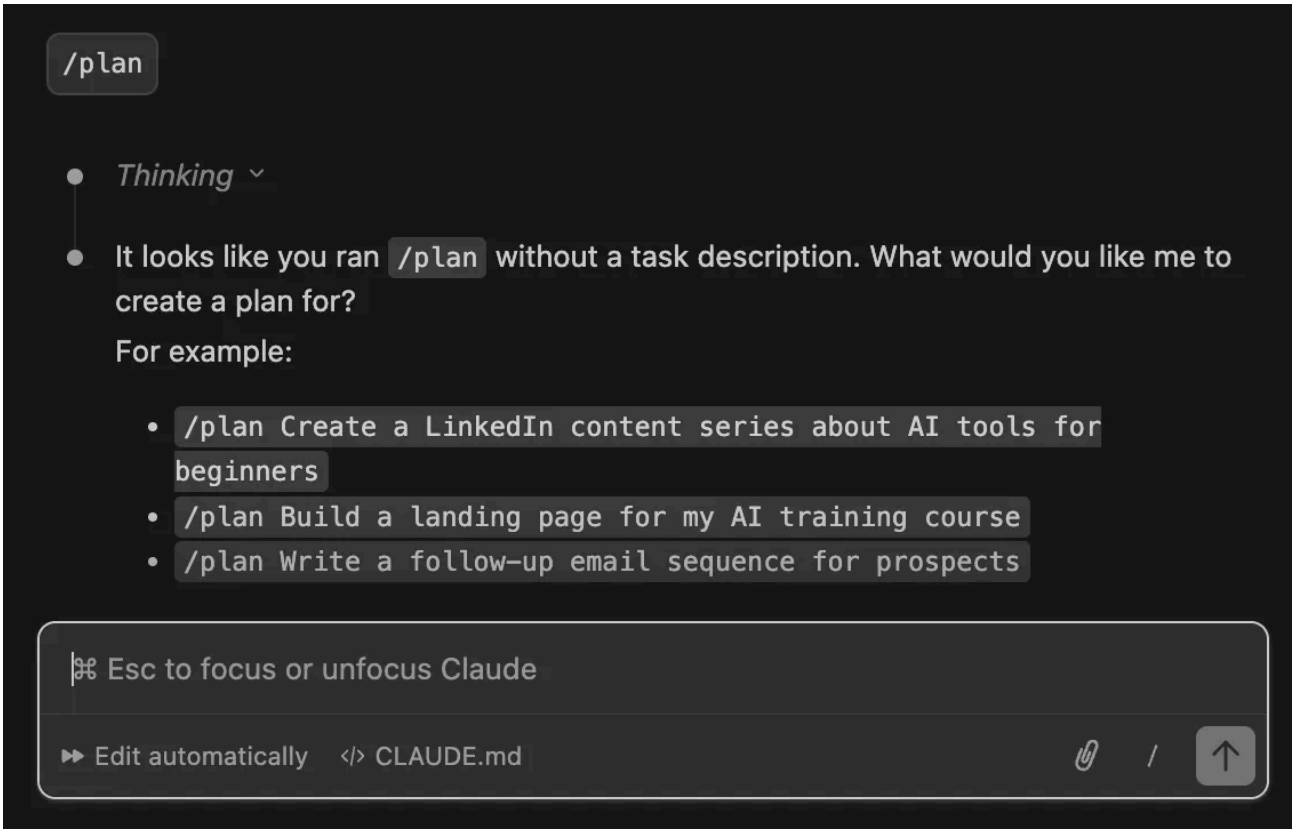
Plan Mode

A minute of planning saves ten minutes of building. That is not motivational language. It is token economics.

Here is the actual maths. You build something without planning. It is wrong. You rebuild. Total cost: build time plus rebuild time plus the tokens consumed by both attempts. You plan first. Claude Code identifies the approach problem before touching a single file. Total cost: five minutes of planning plus one correct build.

Plan Mode is read-only. Claude Code researches your project, reads your files, checks documentation, and produces a plan before executing anything. **The plan is not cosmetic. It is load-bearing.** Every hour I have saved in complex builds came from spending five minutes in Plan Mode first.

How to use it: click “Plan Mode” in the permission toggle at the bottom of the chat panel. Describe what you want to build. Let Claude Code produce the plan. Read it. Push back if something looks off. Switch to Bypass Permissions when the plan looks right, and let it build.



Context management

Context is the scarce resource in Claude Code. Managing it well is the difference between a smooth build and a degraded one.

Slash commands worth knowing

You do not need to memorise all 62.

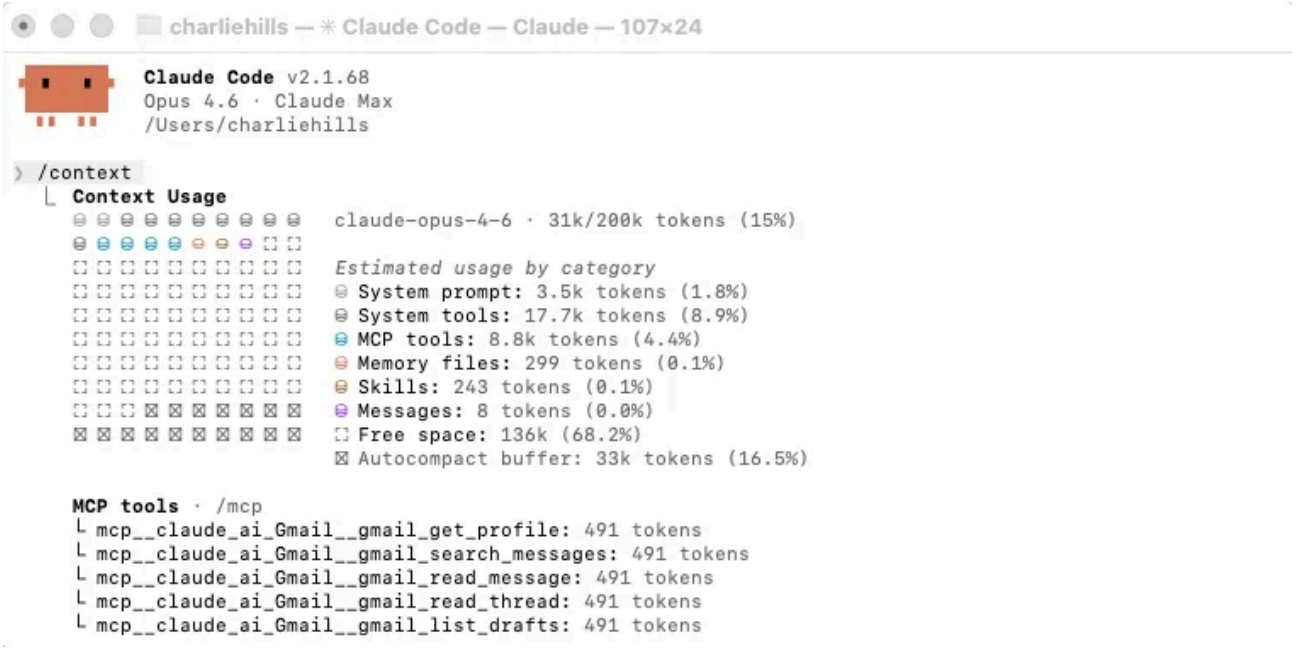
These are the most important ones to learn:

/plan — start Plan Mode before any complex task
/context — see exactly what is consuming your context window
/compact — free up context (do this at around 50%, not 95%)
/clear — reset context entirely when switching to a new task
/model — switch models or reasoning level mid-session

/init — generate your first CLAUDE.md automatically
/doctor — diagnose installation and authentication issues
/rewind — undo when Claude goes off track (or hit Esc twice)
/permissions — pre-allow safe commands instead of using bypass mode

/btw — ask a side question without interrupting the main task

Run `/context` in any session to see exactly what is consuming your context window. You will see your system prompt, tools, MCP tools, memory files, skills, and messages with their exact token counts. Most people are surprised. Before they type a single message, they are often 15-20% through their window from system-level overhead alone.



What you can actually build

This is where Claude Code stops being a productivity tool and starts being a business infrastructure layer.

Each use case below is built as a skill. A skill is a Markdown file that sits in your .claude/skills/ folder. Claude Code reads it on demand and follows it as a step-by-step workflow. You define it once. After that, one line runs the whole thing.

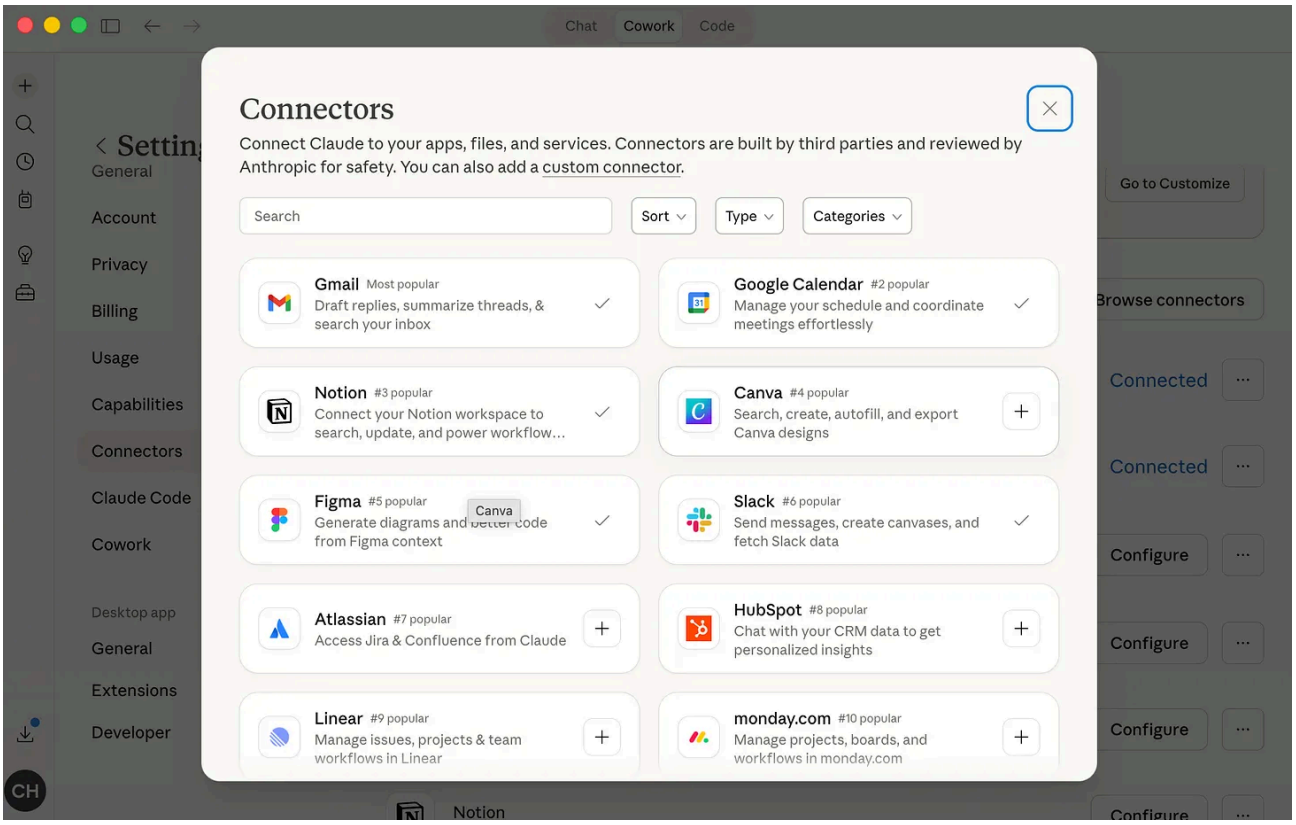
The connector layer

Every skill above gets more useful the moment the right connector is active.

Connect Google Sheets and the lead scraping skill uploads results directly. Connect Gmail and the email classification skill reads and labels your real inbox. Connect Figma and your built interfaces land on the design canvas as editable frames.

The skill provides the workflow. The connector provides the live data and the destination. Without connectors you are working on local files. With them you are running operations inside the tools your business already depends on.

Go to Settings > Connectors in Claude Desktop. Over 50 integrations are listed. Authenticate once. Every future session picks them up automatically.



Four MCP servers worth installing first:

Context7 fetches up-to-date library documentation into context. Stops Claude from hallucinating outdated APIs.

Playwright gives Claude browser automation. It can implement, test, and verify UI features by navigating your app itself.

Excalidraw lets Claude generate architecture diagrams and flowcharts as hand-drawn sketches from a prompt.

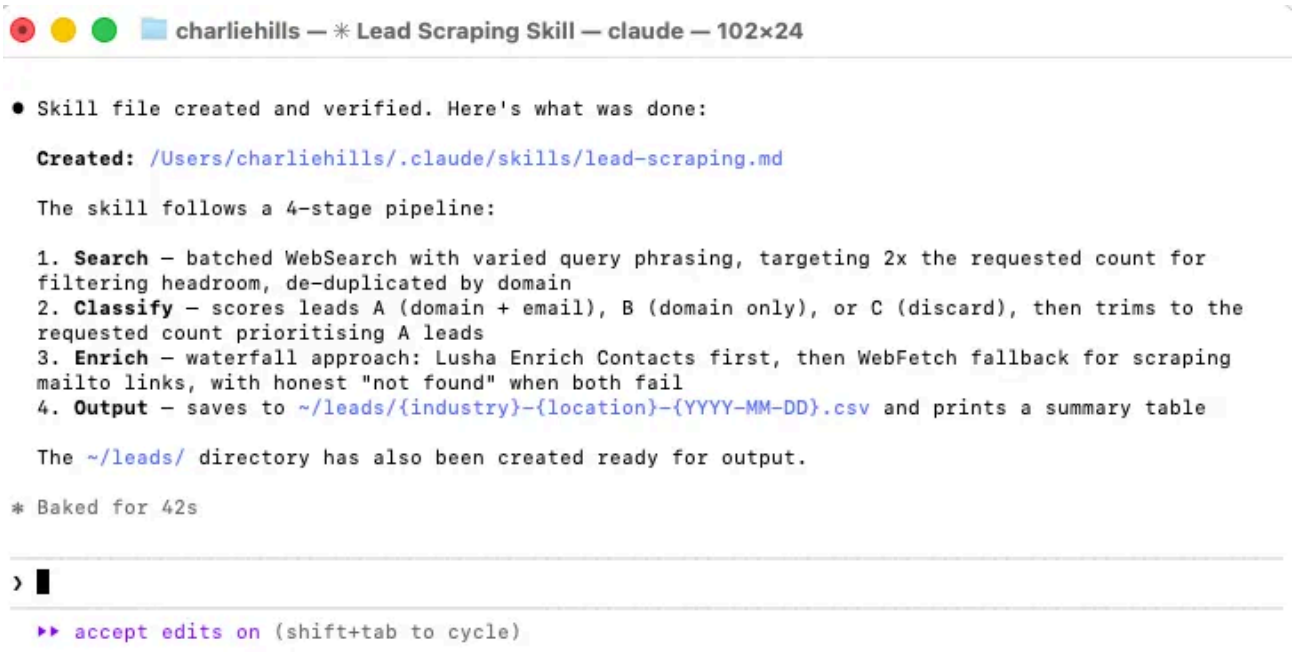
DeepWiki pulls structured documentation for any GitHub repo. Point Claude at a repo and it gets the full picture without you pasting README files into the chat.

Lead scraping

A skill that scrapes a target industry, classifies leads with AI, enriches contact emails, and uploads everything to a Google Sheet.

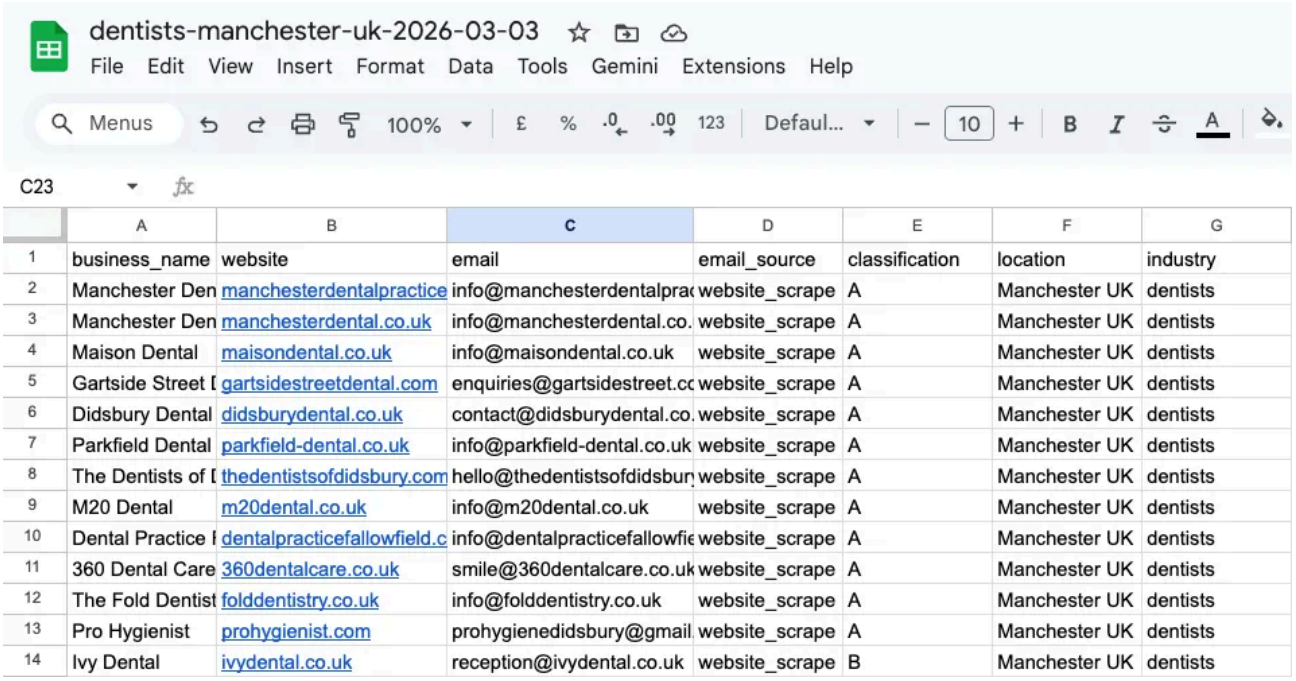
Build the skill. Open Claude Code and paste:

```
Create a Claude Code skill file called lead-scraping.md in a
.claude/skills/ folder.

The skill should:
1. Scrape business leads for a target industry and location using web search
2. Classify each lead with AI (business name, website, phone, category)
3. Enrich a contact email for each lead where publicly available
4. Upload all results to a Google Sheet

Accept three inputs from the user:
- Industry (e.g. dentists, accountants, solicitors)
- Location (e.g. Manchester UK, United States)
- Number of leads required

Write it as a step-by-step Markdown checklist Claude Code follows
when the skill is invoked.
```



Run it with one line:

```
Use the lead-scraping skill.
Industry: dentists
Location: Manchester, UK
Leads required: 20
Upload to this Google Sheet: [YOUR SHEET URL]
```



dentists-manchester-uk-2026-03-03

File Edit View Insert Format Data Tools Gemini Extensions Help

Menus 100% £ % .0 .00 123 Defaul... 10 B I A

C23

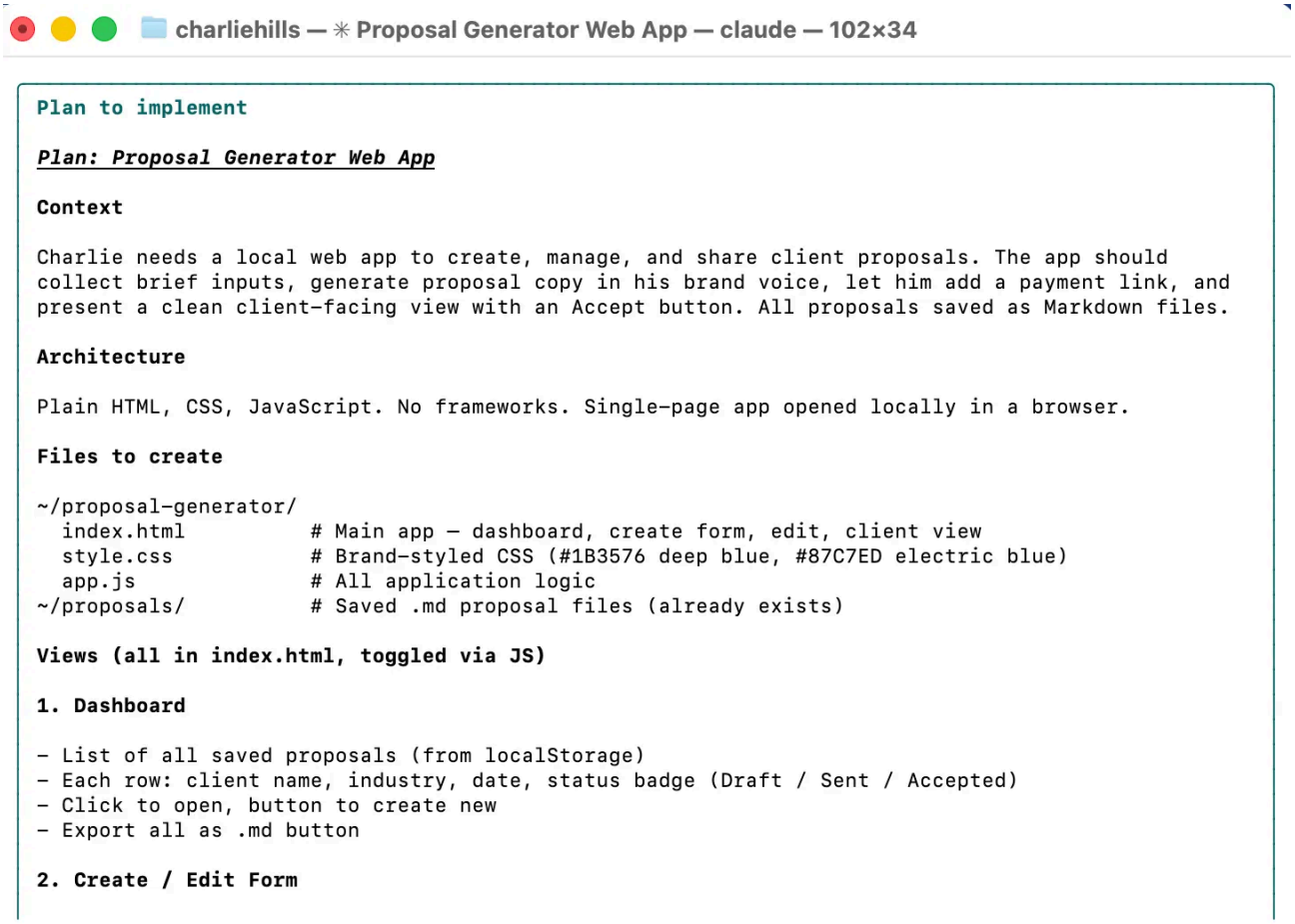
	A	B	C	D	E	F	G
1	business_name	website	email	email_source	classification	location	industry
2	Manchester Den	manchesterdentalpractice	info@manchesterdentalprac	website_scrape	A	Manchester UK	dentists
3	Manchester Den	manchesterdental.co.uk	info@manchesterdental.co.	website_scrape	A	Manchester UK	dentists
4	Maison Dental	maisondental.co.uk	info@maisondental.co.uk	website_scrape	A	Manchester UK	dentists
5	Gartside Street I	gartsidestreetdental.com	enquiries@gartsidestreet.cc	website_scrape	A	Manchester UK	dentists
6	Didsbury Dental	didsburydental.co.uk	contact@didsburydental.co.	website_scrape	A	Manchester UK	dentists
7	Parkfield Dental	parkfield-dental.co.uk	info@parkfield-dental.co.uk	website_scrape	A	Manchester UK	dentists
8	The Dentists of I	thedentistsofdidsbury.com	hello@thedentistsofdidsbur	website_scrape	A	Manchester UK	dentists
9	M20 Dental	m20dental.co.uk	info@m20dental.co.uk	website_scrape	A	Manchester UK	dentists
10	Dental Practice I	dentalpracticefallowfield.c	info@dentalpracticefallowfie	website_scrape	A	Manchester UK	dentists
11	360 Dental Care	360dentalcare.co.uk	smile@360dentalcare.co.uk	website_scrape	A	Manchester UK	dentists
12	The Fold Dentist	folddentistry.co.uk	info@folddentistry.co.uk	website_scrape	A	Manchester UK	dentists
13	Pro Hygienist	prohygienist.com	prohygienedidsbury@gmail	website_scrape	A	Manchester UK	dentists
14	Ivy Dental	ivydental.co.uk	reception@ivydental.co.uk	website_scrape	B	Manchester UK	dentists

Proposal generation

A full proposal generator with a brief input form, AI-generated proposals in your voice, a client-facing view with an Accept button, and a payment link field. No PandaDoc subscription. No per-proposal cost.

Build it using Plan Mode. Switch to Plan Mode first, then paste:

```
/plan Build a proposal generator web app with the following:
- A brief input form (client name, industry, scope, budget)
- AI-generated proposal output written in my voice using my brand-voice.md file
- A client-facing view of the proposal with an Accept button
- A payment link field I can add manually before sending
- Each proposal saved as a Markdown file in a /proposals folder

Use plain HTML, CSS, and JavaScript. No frameworks.
Read all files in this folder before planning.
Ask me clarifying questions before you start building.
```



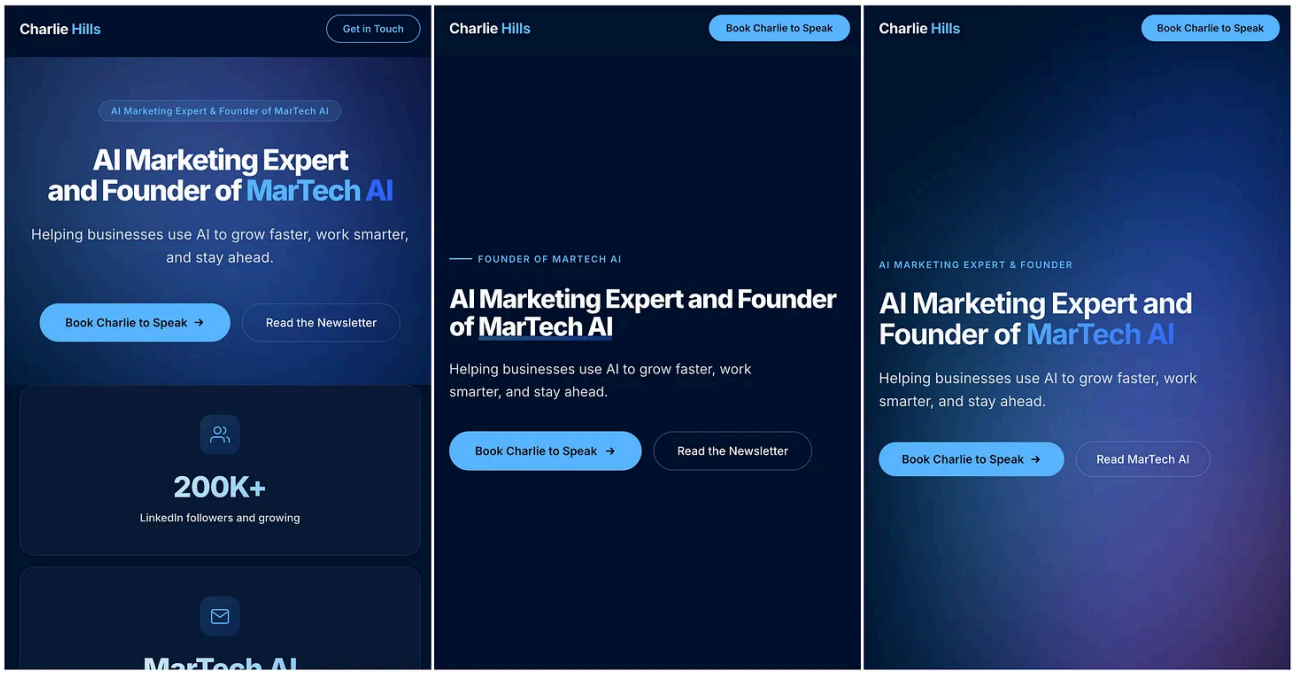
This clears the context window before building so tokens used during planning do not eat into your build. Claude Code then executes every step without stopping to ask permission.

Website generation

Screenshot a site you admire. Feed it to Claude Code alongside your content and brand guidelines. Get a complete, deployable website without touching a template or page builder.

Build it. Drag your reference screenshot into the terminal, then paste:

```
I am going to give you a screenshot of a website I like the design of
and my own content. Build me a complete single-page website using my
content but matching the visual style and layout of the reference.

My content:
Headline: [YOUR HEADLINE]
Subheadline: [YOUR SUBHEADLINE]
Key points: [POINT 1], [POINT 2], [POINT 3]
CTA: [YOUR CTA TEXT]
Brand colour: [HEX CODE]

Build as a single HTML file with embedded CSS.
Produce three layout variants: version-a.html, version-b.html, version-c.html
Run them in parallel.
```



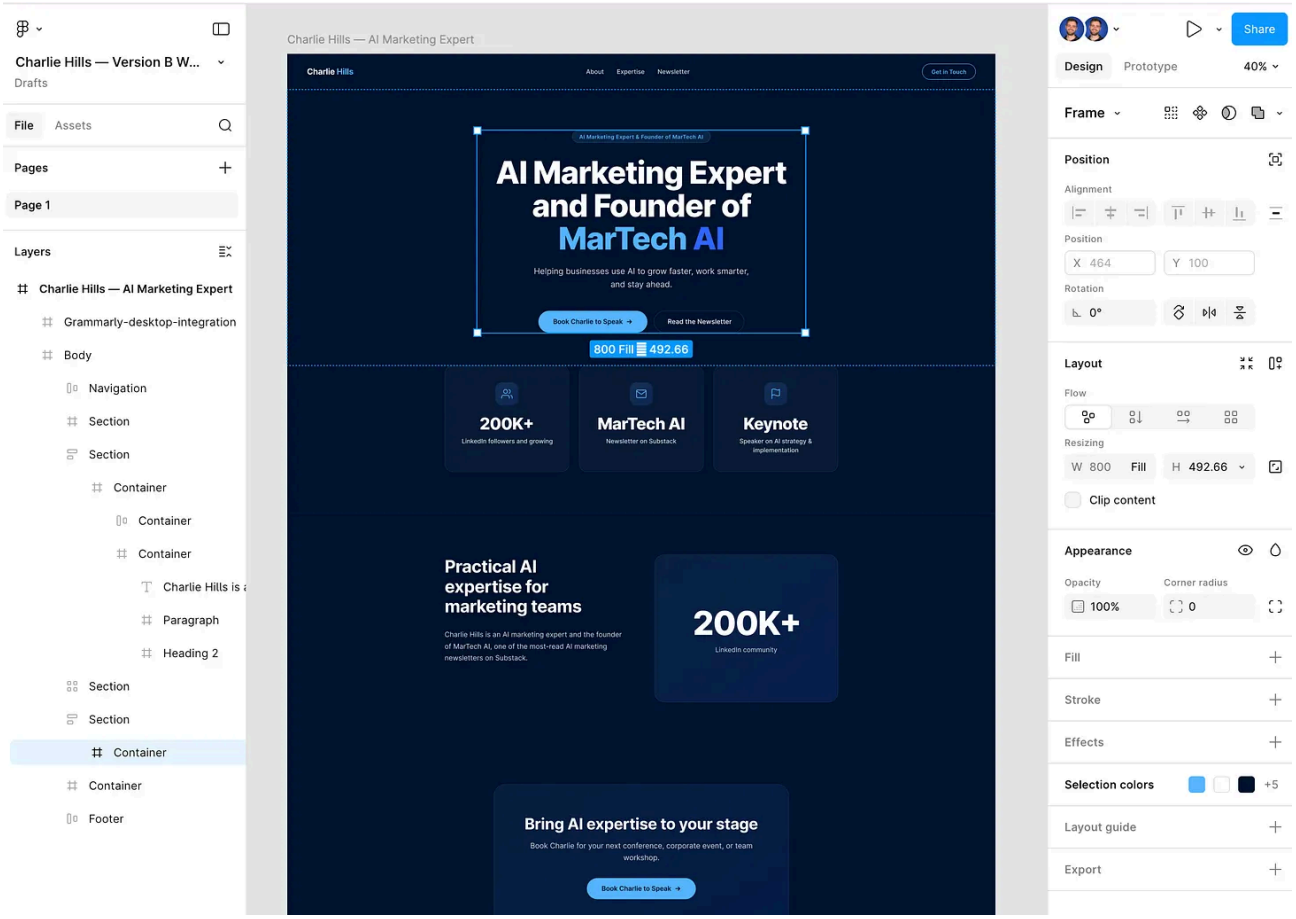
To push the finished build into Figma for design refinement, connect Figma MCP first:

```
claude mcp add --transport http figma https://mcp.figma.com/mcp
```

Then:

```
Start a local server for my app and capture the UI in a new Figma file.
```

Claude Code opens a browser, captures the live interface, and sends it to Figma as editable design layers. Refine it on the canvas, push updates back. Design and code stay in sync without manual handoff.



Video generation

Remotion is an open-source framework that lets you create videos programmatically using React. Every frame is a component. Every animation is code. You describe what you want and Claude Code builds it.

This is not AI generating random video from a text prompt. Claude Code writes the React components that render your video. You get full control, full editability, and the ability to generate variations from a single template.

I used this to build a product launch video for [Vislo, my new AI infographic SaaS.]

Created entirely from the terminal. The first version was rough, and that was down to the quality of the screen recordings I gave it rather than anything Claude Code did wrong. Once I provided better source material and refined the prompts over a few iterations, the output got genuinely good. The fact that you can produce motion graphics from a terminal window is still slightly surreal.

Set it up in four steps.

Step 1. Create a new Remotion project:

```
npx create-video@latest
```

Select the Blank template, say yes to TailwindCSS, and say yes to install Skills when prompted.

Step 2. Navigate into the folder and start the preview server:

```
cd my-video
npm install
npm run dev
```

Step 3. Open a second terminal window and start Claude Code:

```
cd my-video
claude
```

Step 4. Describe the video you want. Claude Code reads your project, writes the components, and the preview at localhost:3000 updates in real time. When you are happy with the result, render your final MP4:

```
npx remotion render
```

The quality of your output depends almost entirely on the quality of your source material. Good screen recordings produce good videos. Blurry screenshots produce generic results. Give Claude Code sharp, high-resolution assets and it handles everything else.

One thing to check before using this commercially. Remotion is free for personal use and open source projects. Companies with three or more employees need a commercial licence to render videos for clients. Check remotion.dev/license before building client work on top of it.

Vislo, the tool I built the demo video for, is live. You describe what you want in plain text, Vislo generates share-ready infographics in seconds (no design skills needed). $19/month and the first 100 users lock in that price. Sign up at https://www.vislo.ai/